TEMA 4 · GRAMÁTICAS INDEPENDIENTES DEL CONTEXTO

Formas normales

Chomsky y Greibach, paso a paso

Basado en el Tema 4 de Serafín Moral, Universidad de Granada.
Dos ejemplos nuevos y su relación con la pertenencia.

Nuria Rodríguez Barroso

Qué significa normalizar una gramática

Imponemos un patrón a todas las producciones para facilitar el trabajo con la gramática.

$$L(G_{\mathrm{Ch}}) = L(G_{\mathrm{Gr}}) = L(G)$$

Cambian las reglas y pueden aparecer variables auxiliares. Las palabras generadas se conservan.

> Trabajaremos con lenguajes que no contienen ε. La palabra vacía se trata mediante una excepción para el símbolo inicial.

Dos patrones para las producciones

Chomsky

$$A \to BC \quad \text{o} \quad A \to a$$

Dos variables o un terminal.

Greibach

$$A \to a\alpha, \quad \alpha \in V^*$$

Un terminal, seguido de cero o más variables.

> B y C son variables. a es un terminal. En Greibach también se permite A → a.

Preparación de la gramática

Símbolos útiles. Eliminamos lo no productivo y después lo inalcanzable.

Producciones nulas. Las eliminamos y tratamos ε por separado.

Producciones unitarias. Las sustituimos por reglas no unitarias.

> Los dos ejemplos ya están preparados: no tienen símbolos inútiles ni producciones nulas o unitarias.

Reglas válidas en Chomsky

Producción	¿Válida?	Motivo
S → AB	Sí	Dos variables
A → a	Sí	Un terminal
S → aB	No	Mezcla terminal y variable
S → ABC	No	Tres variables

Ejemplo 1: la gramática inicial

$$S \to aSb \mid ab \mid c$$

Inicial: S

Terminales: {a, b, c}

Una única variable: S.

> Cada uso de S → aSb añade una a al principio y una b al final.

Qué palabras genera el ejemplo 1

$$L_1 = \{a^n b^n : n \geq 1\} \cup \{a^n c b^n : n \geq 0\}$$

Terminamos con ab

$S \Rightarrow ab$

$S \Rightarrow aSb \Rightarrow aabb$

Terminamos con c

$S \Rightarrow c$

$S \Rightarrow aSb \Rightarrow acb$

> ε no pertenece a este lenguaje. La transformación debe conservar ambas familias de palabras.

Qué tenemos que transformar

$S \to aSb$

$S \to ab$

$S \to c$

aSb: mezcla símbolos y tiene longitud 3.

ab: son dos terminales.

S → c ya es válida.

Primero sustituimos los terminales de las reglas largas. Después reducimos su longitud.

Variables auxiliares para a y b

$$C_a \to a$$
$$C_b \to b$$

C_a representa una a.

C_b representa una b.

Las dos reglas están en Chomsky.

C_a y C_b son variables nuevas. Podemos reutilizarlas en todas las producciones.

Sustituimos los terminales de aSb

$S \to aSb$

$C_a \to a$

$C_b \to b$

a se sustituye por C_a.

b se sustituye por C_b.

$S \to C_aSC_b$

La S central permanece. C_a y C_b recuperarán los terminales originales.

Sustituimos los terminales de ab

$S \to ab$

$S \to c$

$C_a \to a$

$C_b \to b$

S → ab pasa a **$S \to C_a C_b$**.

S → c permanece igual.

No sustituimos c por una variable: S → c ya cumple el patrón de Chomsky.

Solo queda una regla larga

$S \to C_a S C_b \mid C_a C_b \mid c$

$C_a \to a$

$C_b \to b$

$S \to C_a S C_b$ tiene tres variables.

Necesitamos dividirla en reglas con dos variables.

$S \to C_a C_b$, $S \to c$, $C_a \to a$ y $C_b \to b$ ya son válidas.

Dividimos C_aSC_b en dos pasos

$$S \to C_aD_1$$
$$D_1 \to SC_b$$

D_1 es una variable nueva.

$S \Rightarrow C_aD_1 \Rightarrow C_aSC_b$

D_1 agrupa el sufijo SC_b. Ambas producciones tienen exactamente dos variables.

Resultado en forma normal de Chomsky

$$S \to C_a D_1 \mid C_a C_b \mid c$$

$$D_1 \to S C_b$$

$$C_a \to a$$

$$C_b \to b$$

Dos variables:

$S \to C_a D_1$, $S \to C_a C_b$, $D_1 \to S C_b$

Un terminal:

$S \to c$, $C_a \to a$, $C_b \to b$

No hay producciones nulas ni unitarias. Todas las reglas cumplen uno de los dos patrones.

La misma palabra con las nuevas reglas

La palabra **acb** se obtiene en las dos gramáticas.

Antes

$$S \Rightarrow aSb \Rightarrow acb$$

En Chomsky

$$S \Rightarrow C_aD_1 \Rightarrow aD_1 \Rightarrow aSC_b \Rightarrow acC_b \Rightarrow acb$$

> La palabra no cambia. La derivación utiliza más pasos porque las reglas son más pequeñas.

Por qué se conserva todo el lenguaje

Cada aplicación de $S \to aSb$ se simula con $S \to C_a D_1$, $D_1 \to SC_b$, $C_a \to a$ y $C_b \to b$.

$S \to ab$ se simula con $S \to C_a C_b$ y las reglas de C_a y C_b. $S \to c$ no cambia.

En sentido inverso, D_1 solo puede convertirse en SC_b, C_a en a y C_b en b. Las auxiliares no añaden opciones nuevas.

> Comparar una sola palabra ilustra la transformación. La correspondencia entre reglas justifica la equivalencia completa.

El procedimiento de Chomsky

Terminales en reglas largas. Sustituimos cada terminal a por una variable nueva C_a, con la regla $C_a \to a$.

Más de dos variables. Dividimos cada lado derecho largo usando auxiliares nuevas para esa producción.

$$A \to BEFG \quad \rightsquigarrow \quad A \to BD_1,\ D_1 \to ED_2,\ D_2 \to FG$$

> Las reglas que ya son de la forma A → a se conservan.

Reglas válidas en Greibach

Producción	¿Válida?	Motivo
S → aAB	Sí	Terminal seguido de variables
A → a	Sí	Cero variables después
S → AB	No	Empieza por una variable
S → abB	No	El segundo símbolo es terminal

Ejemplo 2: recursión indirecta

$$A1 \to A2A3$$

$$A2 \to A1A3 \mid b$$

$$A3 \to a$$

Inicial: A_1

Orden: A_1, A_2, A_3

Todas las reglas están en Chomsky.

A_1 depende de A_2 y A_2 depende de A_1.

Seguiremos las dos partes del algoritmo. La primera sustitución hará visible la recursión por la izquierda.

Qué lenguaje genera el ejemplo

$$A_1 \Rightarrow A_2A_3 \Rightarrow A_1A_3A_3$$

$$A_1 \Rightarrow A_2A_3 \Rightarrow bA_3 \Rightarrow ba$$

$$L_2 = \{ba^{2k+1} : k \geq 0\}$$

Cada vuelta por A_1 y A_2 añade dos A_3. Para terminar, $A_2 \rightarrow$ b deja una A_3 más.

> Ejemplos: ba, baaa, baaaaa. El número de a es siempre impar.

Las dos operaciones de Greibach

ELIMINA_1(A → Bα). Sustituye esa regla por A → βα para cada producción B → β. Conserva las reglas de B.

ELIMINA_2(A). Elimina la recursión inmediata por la izquierda mediante una auxiliar B_A.

$$A \to A\alpha \mid \beta \quad \rightsquigarrow \quad A \to \beta \mid \beta B_A$$

$$B_A \to \alpha \mid \alpha B_A$$

> En ELIMINA_2, α no es vacía y β no empieza por A.

El orden de las dos partes

Primera parte: k = 1, 2, 3. Para cada k, aplicamos ELIMINA_1 a $A_k \to A_j \alpha$ con $j < k$. Después, si hay $A_k \to A_k \alpha$, aplicamos ELIMINA_2(A_k).

Segunda parte: i = 2, 1. Aplicamos ELIMINA_1 a $A_i \to A_j \alpha$ con $j > i$. Al final sustituimos las variables iniciales en las reglas de B_j.

> $j > i$ es la condición que queda al finalizar la primera parte. En esa primera parte se eliminan los casos $j < i$.

k = 1: revisión de A1

$A1 \to A2A3$

$A2 \to A1A3 \mid b$

$A3 \to a$

$A_1 \to A_2A_3$ tiene $j = 2 > 1$.

ELIMINA_1: no hay $j < 1$.

ELIMINA_2: no hay $A_1 \to A_1\alpha$.

Conservamos $A_1 \to A_2A_3$. Su sustitución corresponde a la segunda parte.

k = 2: ELIMINA_1 primero

$$\text{ELIMINA_1}(A_2 \to A_1 A_3)$$

$A1 \to A2A3$

$A2 \to A2A3A3 \mid b$

$A3 \to a$

j = 1 < 2: sustituimos A_1.

$A_2 \to A_1A_3$ se elimina.

Añadimos **$A_2 \to A_2A_3A_3$**.

$A_2 \Rightarrow A_1A_3 \Rightarrow A_2A_3A_3$. La sustitución revela la recursión inmediata en A_2.

k = 2: después, ELIMINA_2

$$\text{ELIMINA_2}(A_2), \qquad \alpha = A_3A_3, \quad \beta = b$$

$A_1 \to A_2A_3$

$A_2 \to b \mid bB_2$

$A_3 \to a$

$B_2 \to A_3A_3 \mid A_3A_3B_2$

Eliminamos $A_2 \to A_2A_3A_3$.

Conservamos $A_2 \to b$ y añadimos $A_2 \to bB_2$.

B_2 representa una o más parejas A_3A_3.

> La auxiliar es B_2 porque hemos aplicado ELIMINA_2 a A_2. B_2 no genera ε.

k = 3: termina la primera parte

$A_1 \to A_2A_3$

$A_2 \to b \mid bB_2$

$A_3 \to a$

$B_2 \to A_3A_3 \mid A_3A_3B_2$

$A_3 \to a$ ya empieza por un terminal.

No hay llamadas a ELIMINA_1 ni a ELIMINA_2 para A_3.

$A_1 \to A_2A_3$ cumple $2 > 1$.

Ya solo quedan reglas $A_i \to a\alpha$, $A_i \to A_j\alpha$ con $j > i$ y reglas de auxiliares $B_j \to A_i\alpha$.

i = 2, 1: ELIMINA_1 en las A_i

Para i = 2, $A_2 \to b \mid bB_2$ ya empieza por terminal. Para i = 1:

$$\text{ELIMINA_1}(A_1 \to A_2 A_3)$$

$A_1 \to bA_3 \mid bB_2A_3$

$A_2 \to b \mid bB_2$

$A_3 \to a$

$B_2 \to A_3A_3 \mid A_3A_3B_2$

Copiamos las dos alternativas de A_2.

Conservamos el sufijo A_3 en ambas.

$A_1 \to A_2A_3$ desaparece. Ahora todas las producciones de las A_i empiezan por terminal.

ELIMINA_1 en la auxiliar B2

$$\text{ELIMINA_1}(B_2 \to A_3 A_3)$$

$$\text{ELIMINA_1}(B_2 \to A_3 A_3 B_2)$$

$A_3 \to a$

$B_2 \to aA_3 \mid aA_3B_2$

Usamos $A_3 \to a$ en las dos reglas.

Solo sustituimos la primera A_3.

Las variables posteriores al terminal se conservan.

Recorremos i = 1, 2, 3. Solo hay reglas $B_j \to A_i\alpha$ que tratar cuando i = 3.

Resultado en forma normal de Greibach

$A_1 \to bA_3 \mid bB_2A_3$

$A_2 \to b \mid bB_2$

$A_3 \to a$

$B_2 \to aA_3 \mid aA_3B_2$

Cada regla empieza por un terminal.

Después solo aparecen variables.

A_2 queda inalcanzable: se puede eliminar.

B_2 genera un número positivo y par de a. Desde A_1 obtenemos b seguida de un número impar de a.

La misma palabra: baaa

Gramática original en Chomsky

$$A_1 \Rightarrow A_2A_3 \Rightarrow A_1A_3A_3 \Rightarrow A_2A_3A_3A_3$$
$$\Rightarrow bA_3A_3A_3 \Rightarrow baA_3A_3 \Rightarrow baaA_3 \Rightarrow baaa$$

Gramática en Greibach

$$A_1 \Rightarrow bB_2A_3 \Rightarrow baA_3A_3 \Rightarrow baaA_3 \Rightarrow baaa$$

> Con n = 4: la derivación en Chomsky tiene 7 pasos y la derivación en Greibach tiene 4.

Las llamadas del ejemplo

Primera parte, k = 2. ELIMINA_1($A_2 \to A_1A_3$) y después ELIMINA_2(A_2). Para k = 1 y k = 3 no hay transformaciones.

Segunda parte, i = 1. ELIMINA_1($A_1 \to A_2A_3$), usando las dos alternativas de A_2.

Auxiliar, i = 3. ELIMINA_1($B_2 \to A_3A_3$) y ELIMINA_1($B_2 \to A_3A_3B_2$).

> En la primera parte, ELIMINA_1 y ELIMINA_2 se revisan para cada k antes de pasar al siguiente.

Ejemplo 3: sin pasar por Chomsky

$A_1 \to A_2A_3A_3 \mid a$

$A_2 \to A_1A_3 \mid bA_3$

$A_3 \to c$

Inicial: A_1

Orden: A_1, A_2, A_3

$A_1 \to A_2A_3A_3$ tiene tres variables.

$A_2 \to bA_3$ mezcla terminal y variable.

No está en Chomsky. Sí cumple las condiciones de entrada: tras el primer símbolo solo hay variables, sin reglas nulas ni unitarias.

El lenguaje y el orden de trabajo

$$L_3 = \{a\} \cup \{xc^{3k} : x \in \{a, b\},\ k \geq 1\}$$

$A_1 \to a$ genera a. Al salir por A_2, obtenemos a o b seguidas de tres c. Cada vuelta $A_1 \Rightarrow A_2A_3A_3 \Rightarrow A_1A_3A_3A_3$ añade otras tres.

Primera parte. k = 1, 2, 3. ELIMINA_1 con j < k y después ELIMINA_2 si hay recursión.

Segunda parte. i = 2, 1. ELIMINA_1 con j > i y, al final, en las auxiliares.

k = 1: las reglas de A1 se conservan

$A1 \rightarrow A2A3A3 \mid a$

$A2 \rightarrow A1A3 \mid bA3$

$A3 \rightarrow c$

$A_1 \rightarrow A_2A_3A_3$ tiene j = 2 > 1.

$A_1 \rightarrow$ a empieza por terminal.

No hay llamadas a ELIMINA_1 ni a ELIMINA_2.

La longitud del lado derecho no obliga a transformarlo a Chomsky para aplicar este algoritmo.

k = 2: ELIMINA_1 con dos alternativas

$$\text{ELIMINA_1}(A_2 \to A_1 A_3)$$

$A1 \to A2A3A3 \mid a$

$A2 \to A2A3A3A3 \mid aA3 \mid bA3$

$A3 \to c$

j = 1 < 2. Usamos ambas reglas de A_1.

$A_1 \to A_2A_3A_3$ aporta $A_2 \to A_2A_3A_3A_3$.

$A_1 \to a$ aporta $A_2 \to aA_3$.

El sufijo A_3 de la producción sustituida se añade a cada alternativa. $A_2 \to bA_3$ se conserva.

k = 2: ELIMINA_2 con dos salidas

$$\alpha = A_3A_3A_3, \qquad \beta_1 = aA_3, \quad \beta_2 = bA_3$$

$A_1 \to A_2A_3A_3 \mid a$

$A_2 \to aA_3 \mid bA_3 \mid aA_3B_2 \mid bA_3B_2$

$A_3 \to c$

$B_2 \to A_3A_3A_3 \mid A_3A_3A_3B_2$

ELIMINA_2(A_2). Creamos B_2.

Para cada salida β conservamos $A_2 \to \beta$ y añadimos $A_2 \to \beta B_2$.

B_2 repite bloques de tres A_3.

Eliminamos $A_2 \to A_2A_3A_3A_3$. Ninguna producción nueva genera ε.

k = 3: fin de la primera parte

$A_1 \to A_2A_3A_3 \mid a$

$A_2 \to aA_3 \mid bA_3 \mid aA_3B_2 \mid bA_3B_2$

$A_3 \to c$

$B_2 \to A_3A_3A_3 \mid A_3A_3A_3B_2$

$A_3 \to c$ no requiere cambios.

$A_1 \to A_2A_3A_3$ cumple $j = 2 > 1$.

Las reglas de A_2 ya empiezan por terminal.

> Solo falta sustituir A_2 al principio de la regla de A_1 y A_3 al principio de las reglas de B_2.

ELIMINA_1 en A1: cuatro alternativas

i = 2 no requiere cambios. Para i = 1 aplicamos ELIMINA_1($A_1 \to A_2A_3A_3$).

$A1 \to a$

$A1 \to aA3A3A3 \mid bA3A3A3$

$A1 \to aA3B2A3A3 \mid bA3B2A3A3$

A_2 tiene cuatro alternativas.

A cada una añadimos el sufijo A_3A_3.

La regla $A_1 \to a$ se conserva.

> Las producciones de A_2, A_3 y B_2 permanecen como al final de la primera parte.

ELIMINA_1 en las reglas de B2

$$\text{ELIMINA_1}(B_2 \to A_3 A_3 A_3)$$

$$\text{ELIMINA_1}(B_2 \to A_3 A_3 A_3 B_2)$$

$A_3 \to c$

$B_2 \to cA_3A_3 \mid cA_3A_3B_2$

Al recorrer i = 1, 2, 3, solo actuamos para i = 3.

Cada regla empieza ahora por c.

Las dos A_3 posteriores se conservan.

Todas las producciones cumplen ya el patrón de Greibach.

Resultado sin pasar por Chomsky

$$A_1 \to a$$

$$A_1 \to aA_3A_3A_3 \mid bA_3A_3A_3$$

$$A_1 \to aA_3B_2A_3A_3 \mid bA_3B_2A_3A_3$$

$$B_2 \to cA_3A_3 \mid cA_3A_3B_2$$

$$A_3 \to c$$

Inicial: A_1.

Eliminamos A_2 porque ha quedado inalcanzable.

B_2 genera 3, 6, 9, … letras c.

Se conserva $L_3 = \{a\} \cup \{xc^{3k} : x \in \{a, b\}, k \geq 1\}$.

La misma palabra: bccc

Gramática original

$$A_1 \Rightarrow A_2A_3A_3 \Rightarrow bA_3A_3A_3 \Rightarrow bcA_3A_3 \Rightarrow bccA_3 \Rightarrow bccc$$

Gramática en Greibach

$$A_1 \Rightarrow bA_3A_3A_3 \Rightarrow bcA_3A_3 \Rightarrow bccA_3 \Rightarrow bccc$$

> Greibach necesita 4 pasos para esta palabra de longitud 4. La gramática original no está en Chomsky: no se le aplica la fórmula 2n − 1.

Pertenencia: profundidad de búsqueda

Para una palabra de longitud n ≥ 1, contamos aplicaciones de producciones.

Chomsky

$$2n - 1$$

n reglas a terminales y n − 1 reglas binarias.

Greibach

$$n$$

Cada aplicación introduce exactamente un terminal.

> Es la profundidad de la búsqueda de derivaciones. No es la altura del árbol de derivación.

Por qué Chomsky sigue siendo útil

$$A \to BC, \qquad w = uv$$

Para saber si A genera w, buscamos una partición donde B genere u y C genere v.

El algoritmo CYK guarda qué variables generan cada fragmento y reutiliza esos resultados.

La estructura binaria permite programación dinámica, sin enumerar todas las derivaciones.

> Greibach reduce la profundidad de una búsqueda exhaustiva. Eso no garantiza menor tiempo de ejecución.

Un paso de CYK con el ejemplo 1

Para reconocer **acb** con la gramática en Chomsky:

Fragmento	Variable	Justificación
a	C_a	$C_a \to a$
c	S	$S \to c$
b	C_b	$C_b \to b$
cb	D_1	$D_1 \to SC_b$ combina c y b
acb	S	$S \to C_aD_1$ combina a y cb

> S genera el fragmento completo: acb pertenece al lenguaje.

La palabra vacía como excepción

Las formas descritas no pueden generar ε: ninguna producción elimina todos los símbolos.

Si ε pertenece al lenguaje original, se permite una excepción $S_0 \to \varepsilon$ con un inicial nuevo que no aparezca en lados derechos.

> Las demás producciones mantienen el patrón correspondiente. Las cotas n y 2n − 1 se aplican a palabras no vacías.

Qué cambia y qué se conserva

Aspecto	Chomsky	Greibach
Regla	$A \to BC$ o $A \to a$	$A \to a\alpha$, con $\alpha \in V^*$
Estructura	Descomposición binaria	Un terminal por paso
Uso destacado	Algoritmo CYK	Derivaciones y autómatas con pila
Lenguaje	Se conserva, con ε tratada por separado	

> Una forma normal no garantiza que la gramática sea no ambigua.

RESUMEN

Las ideas que nos llevamos

Chomsky. Sustituimos terminales de reglas largas y dividimos lados derechos en pares.

Greibach. Eliminamos recursión por la izquierda y sustituimos variables iniciales hasta empezar por un terminal.

Pertenencia. Distinguimos la longitud de una derivación de la eficiencia del algoritmo que la busca.

Referencia conceptual: Serafín Moral, Tema 4, Universidad de Granada. Ejemplos nuevos y desarrollo docente: Nuria Rodríguez Barroso.